

RakshakAI: A Security-First Developer Workflow for Local Vulnerability Triage and Evidence-Aware Reporting

RakshakAI Project

Open-source security engineering prototype

<https://github.com/Muneerali199/RakshakAI>

Abstract—Developers need security feedback where they write code, but a useful tool must also make its evidence, limitations, and remediation path visible. This paper presents RakshakAI, an open-source prototype that combines lightweight static checks, configurable language-model analysis, a local FastAPI service, a Visual Studio Code extension, and a Notion reporting workflow. The system produces structured findings containing a CWE identifier, severity, confidence, root cause, attack scenario, remediation guidance, patched code, and references. We describe the end-to-end design and the data-governance pipeline used for future security-model fine-tuning. We also report the project’s currently verified model evaluation without extrapolation: at checkpoint 375 of 750 training steps, the evaluator detected 7 of 8 manually curated vulnerable snippets (87.5%). This small evaluation does not estimate false-positive rate, generalization to real CVEs, or multi-file performance. By making those boundaries explicit, RakshakAI offers a reproducible foundation for IDE-centered vulnerability triage rather than unsupported claims of production readiness.

Index Terms—software security, vulnerability detection, developer tooling, static analysis, large language models, secure software development

I. Introduction

Security feedback often arrives after code review or CI, when the cost of remediation is high. Static Application Security Testing (SAST) can move feedback earlier, yet rule-only systems can be difficult to interpret and model-only systems can be costly, opaque, or unsuitable for private code. Recent work has shown both the promise and security risks of code-capable language models [1], [2]. The operational question is therefore not simply whether a model can label a snippet, but whether a developer can inspect, act on, and audit the resulting finding.

RakshakAI is an engineering prototype for that workflow. It is designed to keep the developer-facing interaction local: a VS Code extension calls a local FastAPI service, surfaces findings as diagnostics, and can export complete reports to a shared Notion Security Center. The service supports multiple analysis providers, including locally hosted options, and uses a structured output contract rather than free-form prose.

This paper makes three contributions:

- 1) an implementation-oriented architecture for IDE, API, and reporting integration;

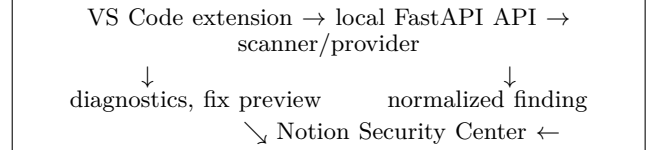


Fig. 1. RakshakAI’s local developer workflow.

- 2) a structured vulnerability-report schema that preserves actionable evidence through the workflow; and
- 3) a transparent account of the current evaluation and dataset readiness, including what has not yet been measured.

II. System Goals and Threat Model

RakshakAI is intended to reduce time-to-triage for source-level vulnerabilities. Its primary asset is developer source code; its primary user is a developer or security reviewer operating inside a repository. The system aims to identify suspicious code and supply a reviewable explanation. It is not an exploit-prevention boundary and it must not be treated as proof that code is safe.

The design follows four goals: (i) fast feedback in the editor, (ii) structured rather than narrative findings, (iii) provider and deployment flexibility, and (iv) auditable sharing of findings. Inputs from a model are treated as untrusted analysis: a human reviews the finding and generated patch before applying it. The extension HTML-escapes explanatory content and restricts rendered external links to HTTP(S), preventing a model-generated reference from becoming an executable webview URL.

III. Architecture

Figure 1 summarizes the data flow. The extension sends the active source document, language, and selected provider to the local service. The service performs lightweight checks and provider-backed analysis, normalizes the response, and returns a typed finding. The extension then renders diagnostics, details, and code actions. On export, it transmits the complete evidence bundle to the Notion API route, which creates one report page per finding.

A. VS Code Extension

The TypeScript extension contributes a left Activity Bar container, a security dashboard, a findings tree, editor diagnostics, and commands for scanning, fixing, and export. It uses cancellation controllers for in-flight document scans and refreshes the sidebar after scan completion so export controls reflect current findings. The extension communicates with the service using HTTPS-capable Axios requests; the local development default is <http://localhost:8080>.

B. API and Analysis Layer

The backend is implemented with FastAPI. Scan and fix routes accept code, language, provider, and optional model selection. A scanner module contains deterministic patterns for high-signal categories such as hard-coded credentials, SQL string construction, and path traversal, while provider adapters can request deeper analysis. The normalized result uses the fields shown in Listing 1. A confidence value is a model or heuristic signal, not a calibrated probability guarantee.

Listing 1. Normalized vulnerability finding

```
{
  "vulnerability": "SQL injection",
  "cwe": "CWE-89",
  "severity": "high",
  "confidence": 0.96,
  "root_cause": "Untrusted input enters a query",
  "attack_scenario": "Crafted identifier changes query logic",
  "secure_fix": "Use parameter binding",
  "patched_code": "...",
  "references": ["https://cwe.mitre.org/..."]
}
```

C. Evidence-Aware Notion Export

The Notion integration is deliberately a reporting layer, not merely a link dump. It creates one page for each exported finding and includes severity, status, CWE, language, repository, path, line number, vulnerable snippet, patched snippet, remediation guidance, references, and a remediation checklist. The batch endpoint validates severity and bounded request fields before page creation. When Notion is not available, individual reports can be queued; batch export instead returns a clear configuration error so the UI does not falsely claim success.

IV. Data Engineering for Future Fine-Tuning

RakshakAI includes a versioned pipeline for security instruction data. It enforces a nine-field security sample schema, cleans exact and near duplicates, checks for harmful-content and license-policy violations, balances long-tail CWE classes, and builds a locked evaluation set. At the reported Phase 2 checkpoint, the cleaned source set contained 112 unique samples; balancing produced 270 examples; and the instruction generation stage produced 549 training, 8 validation, and 9 test records. A separate 31-sample benchmark was pinned with a SHA-256 lock.

TABLE I

Verified checkpoint-375 results. Small denominators require caution.

Category	Correct	Total	Detection rate
All vulnerable snippets	7	8	87.5%
SQL injection	2	2	100.0%
Cross-site scripting	1	2	50.0%
Other categories	4	4	100.0%

These quantities describe pipeline artifacts, not a claim that a production-quality foundation model has been trained.

The project recommends a 7B base model for the current data scale. This is a capacity-control decision: scaling to a 14B model before collecting more diverse vulnerable/fixed pairs would increase overfitting risk without establishing a stronger evaluation baseline. The recommended next step is to ingest real CVE-bearing datasets, preserve provenance, and evaluate only on held-out, adversarially reviewed samples.

V. Evaluation

A. Protocol

The only result presented as measured model performance comes from the project’s checkpoint-375 evaluation. It contains eight manually curated vulnerable snippets covering SQL injection, cross-site scripting, buffer overflow, command injection, format-string misuse, and code injection. The evaluator was at 375 of 750 intended training steps. Table I reports raw counts rather than extrapolated population metrics.

The missed XSS case used partial string replacement before assigning data to innerHTML; it remained exploitable through alternative payloads. This failure is useful because it exposes a concrete training and benchmark gap: sanitization-bypass examples must be represented explicitly.

B. What the Results Do Not Establish

No clean-code examples were included in this eight-sample run, so false-positive rate, precision on a realistic codebase, and F1 cannot be inferred. The study does not evaluate real CVE generalization, calibration, multi-file vulnerabilities, adversarial prompt resistance, or provider-to-provider variability. Claims such as “beats GPT-4,” 90%+ real-world accuracy, or zero false positives are therefore out of scope. This limitation is central to the contribution: an honest benchmark report is more useful to engineering decisions than an unrepeatable leaderboard.

VI. Reproducibility and Operational Use

The repository contains the FastAPI service, scanner, extension source, Notion routes, tests, and benchmark guidance. A local run starts the service and points the extension’s `serverUrl` and `notionServerUrl` settings to it. Notion export requires an explicitly configured integration

token and database; credentials are read from environment variables and are not embedded in the extension.

For operational use, teams should record the model/provider, prompt or rules version, scan timestamp, source revision, and reviewer disposition with each finding. A generated patch should be presented as a diff and verified by tests before merge. These controls make the workflow useful even when a finding is later marked false positive or accepted risk.

VII. Limitations and Future Work

RakshakAI is an evolving prototype. Its present evaluation is small, its detector is principally single-file, and its structured analysis can vary by provider. The Notion integration stores security findings in a third-party workspace and should be enabled only under an organization’s data-handling policy. Future work will (i) expand the held-out benchmark with clean and adversarial samples, (ii) report precision, recall, F1, calibration, and latency with confidence intervals, (iii) add multi-file and dependency-aware analysis, and (iv) evaluate generated patches using regression tests and secure-code review.

VIII. Conclusion

RakshakAI demonstrates a practical local workflow for moving vulnerability triage into the editor while preserving the evidence required for review and reporting. Its principal result is not a claim of security-model supremacy: it is a transparent, reproducible implementation that couples diagnostics, structured findings, patch review, and auditable Notion reports. The preliminary 7/8 result motivates further study, but the project’s next milestone is rigorous evaluation rather than larger marketing numbers.

References

- [1] H. Pearce et al., “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in IEEE Symposium on Security and Privacy, 2022.
- [2] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.
- [3] MITRE, “Common Weakness Enumeration,” <https://cwe.mitre.org/>.
- [4] OWASP Foundation, “OWASP Top 10,” <https://owasp.org/www-project-top-ten/>.
- [5] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>.